컴파일러 수업 정리 노트 (Ch1~3)

Ch1. Introduction — 컴파일러 개론

1.1 컴파일러의 정의

컴파일러 = 번역기(Translator). 소스 언어(source language)로 작성된 입력 문장들을 타겟 언어(target language)의 동등한(equivalent) 문장으로 변환하는 프로그램이다.

1.2 프로그램 실행의 세 가지 방식

(1) 컴파일러 방식

소스 프로그램(C/C++)을 타겟 프로그램(기계어 실행 파일, executable)으로 변환 후, HW(CPU)에서 직접 실행한다. Compile time과 Runtime이 분리된다.

```
[C/C++ source]                      [machine-language executable]
for (j=0; j<i; j++) {     컴파일    $006
  if (A[j]>A[j+1]) {   ------->    LDWS -4(%r23),%r25
    temp = A[j];                    LDWS,MA 4(%r23),%r26
    A[j] = A[j+1];                 COMB,<= %r25,%r26,$007
    A[j+1] = temp;                 STWS %r26,-8(%r23)
  }                                 STWS %r25,-4(%r23)
}                                   $007
                                    ADDIB,<,N 1,%r24,$006+4
                                    LDWS -4(%r23),%r25
```

(2) 인터프리터(Virtual Machine) 방식

별도의 컴파일 없이, 인터프리터(SW)가 소스 프로그램을 직접 실행한다. Python, JavaScript 등. compile time 없이 runtime만 존재. 컴파일 방식보다 느리지만 개발이 빠르고 에러 핸들링과 이식성(portability)이 좋아 현재 주류(mainstream)이다.

(3) 하이브리드 방식 (Compiler + Virtual Machine)

Java가 대표. 소스를 IR(바이트코드)로 컴파일한 후, VM에서 실행한다. VM은 인터프리터 또는 JIT(Just-In-Time) 컴파일러를 사용한다. 이식성과 성능을 동시에 추구.

```
[Java source] → 컴파일러 → [bytecode IR] → VM(인터프리터/JIT) → 결과
   compile time                                  runtime
```

바이트코드 예시: `sipush 1000 / newarray int / astore_1 / iconst_0 / istore_2 / goto 18 / ...`

1.3 현대 컴파일러의 구조

컴파일은 (1) 소스 언어의 분석(analysis)과 (2) 타겟 언어의 합성(synthesis)으로 이루어진다.

```
                    FRONT-ENDs (Analysis)
C++ source ──┐
Fortran src ─┤→ Scanner(Lexer) ↔ Symbol Tables
C source ────┘   Parser ↔ Code Generator
                              ↓
                    MIDDLE-ENDs
              Intermediate Code → Optimization
                              ↓
                    BACK-ENDs (Synthesis)
              Code Generator → Low-Level IR → Optimizer
                              ↓
              Relocatable Object Code (foo.o, boo.o)
                              ↓
                    RUN-TIME ARCHITECTURE
              Linker (+ libc.a, libc.sl) → executable (a.out)
              Loader (dld.sl) → Computer → Input/Output
```

Front-end (분석): - Lexical analysis (어휘 분석) - Syntactic analysis (구문 분석) - Semantic analysis (의미 분석)

Back-end (합성): - IR 생성: P-code, U-code, bytecode, parse tree, AST, register-based IR 등 - Machine-independent optimization (기계 독립 최적화) - Machine code generation and optimization (기계 종속 코드 생성 및 최적화)

Runtime architecture: Linking, Loading, Shared libraries (DLL)

LLVM (Low Level Virtual Machine)

모듈화되고 재사용 가능한 컴파일러 및 툴체인 모음. GCC(GNU Compiler Collection)와 유사.

```
C       → Clang C/C++/ObjC Frontend ─┐                    ┌→ LLVM
X86 Backend → X86
Fortran → llvm-gcc Frontend          ├→ LLVM Optimizer ──→├→ LLVM
PowerPC BE  → PowerPC
Haskell → GHC Frontend               ─┘   (LLVM IR)       └→ LLVM
ARM Backend → ARM
```

3단계 구조: Source Code → Frontend → **LLVM IR** → Optimizer → **LLVM IR** → Backend → Machine Code

1.4 프론트엔드 복습

Lexical Analysis (Lexer)

입력 문자 스트림을 하나씩 읽어 사전 정의된 토큰(token)을 식별한다. 정규 표현식(Regular Expression)으로 토큰 패턴을 정의하고, 유한 오토마타(Finite Automata) 기반 Lexer를 자동 생성한다.

도구: `lex foo.l; cc lex.yy.c -ll; a.out < test.c;`

lex 파일 구조 — (정규 표현식, 액션) 쌍:

```
Letter  [a-zA-Z]
Digit   [0-9]
Id      {Letter}({Letter}|{Digit})*
%%
{Id}    {yylval = install_id(); return(ID); }
%%
```

```
install_id { ... }
```

Syntax Analysis (Parser)

토큰에서 문법 구조를 복원하여 파스 트리(parse tree)를 구성한다. 문맥 자유 문법(Context-Free Grammar)으로 문법을 표현한다.

프로덕션(production) 예시:

```
var_decl : type ID ";"
         | type ID "[" const_expr "]" ";"
type     : type_id | struct_specifier
type_id  : ID
```

`int x ;` → 파스 트리: var_decl → type(type_id(ID:“int”)) + ID:“x” + “;”

파싱 방식: - Top-down parsing: 시작 심볼에서 아래로 확장 (도구: Ometa) - Bottom-up parsing: 잎에서 시작 심볼로 구축 — **더 강력함** (도구: yacc)

도구를 사용해 파서를 자동 생성한다 → “**컴파일러는 컴퓨터 과학의 승리 (triumph)**”

Semantic Analysis (의미 분석)

문맥 자유 문법으로 표현 불가능한 규칙을 검사한다: - 변수를 사용 전에 선언했는가 (e.g., `int x; ..., x = 10;`) - 실인자와 형식인자의 개수가 일치하는가 - 타입이 일치하는가 (e.g., `x = y + z;`에서 정수+문자열 방지)

심볼 테이블(symbol table)을 구축하여 분석한다. **자동 생성 불가** — 직접 구현 필요. 효율을 위해 파싱 도중 의미 분석과 코드 생성을 동시에 수행할 수 있다.

파싱 중 의미 분석 예시 (yacc 문법+액션):

```
var_decl : type ID ";" { declare($2, makevardecl($1)); }
unary    : INT-CONST   { $$ = makenumconstdecl(inttype, $1); }
         | ID          { $$ = findcurrentdecl($1); }
binary   : unary       { $$ = $1->type; }
         | binary '+' binary { $$ = plustype($1, $3); }
assignment : unary "=" expr { check_isvar($1);
check_compatible($1,$3); $$ = $1->type; }
```

Intermediate Representation (IR)

IR의 종류: - **Parse tree / AST**: 트리 구조 - **Abstract stack machine code**: `push a; push b; add; store c` - `x = y;` (x: 전역, y: 지역)의 스택코드: `push_const Lglob+4, push_reg SP, push_const 1, add, fetch, assign` - **Register-based intermediate code**: 가상 레지스터 사용

스택 머신 코드의 장점: 코드 생성이 단순하고, 코드 크기가 작다.

1.5 컴파일러 최적화

최적화의 정의

> 계산을 동등하지만 더 나은**(equivalent but better)** 계산으로 변환하는 것

실행 시간 공식

```
실행 시간 = Instruction Count × CPI × Cycle Time
```

- Instruction Count (IC): 실행되는 명령어의 총 수
- CPI (Cycles Per Instruction): 명령어 하나당 평균 클럭 사이클 수
- Cycle Time: 클럭 한 사이클의 시간 (HW에 의해 결정)

컴파일러가 줄일 수 있는 것: IC와 CPI (Cycle Time은 HW의 몫)

구체적 최적화 기법 목록

- 코드 내 명령어 수 감소
- 레지스터 할당 개선
- 독립 명령어 그룹핑 (슈퍼스칼라 병렬 실행)
- 비싼 명령어를 싼 명령어로 교체 (e.g., multiply → add/shift)
- 루프를 더 밀도 있게(denser) 만듦
- 멀티코어 병렬화
- 캐시 미스 감소

메모리 계층별 지연 시간 (Numbers Everyone Should Know)

```
L1 cache reference           0.5 ns
Branch mispredict              5 ns
L2 cache reference            10 ns
Mutex lock/unlock            100 ns
Main memory reference        100 ns       ← L1 대비 200배
Compress 1KB (zippy)      10,000 ns
Send 2KB over 1Gbps       20,000 ns
Read 1MB from memory     250,000 ns
RTT within datacenter    500,000 ns
1 disk seek           10,000,000 ns       ← L1 대비 2천만배
Read 1MB from disk    30,000,000 ns
```

→ 컴파일러가 데이터를 레지스터/캐시에 잘 유지하도록 최적화하면 성능 차이가 극적.

최적화가 ISA 설계에 미치는 영향

IBM PowerPC, HP PA-RISC 등의 ISA에는 컴파일러 최적화를 전제로 설계된 **복합 명령어(compound instruction)**가 있다.

예: 배열 초기화 `for (i=0; i<N; i++) A[i]=0;`의 최적화된 어셈블리:

```
LDO   -440(%r30),%r31        ; &A
LDI   -100,%r23              ; 카운터 = -100
$003
ADDIB,< 1,%r23,$003          ; 카운터++, 카운터<0이면 $003으로
STWM  %r0,100(%r31)          ; 0 → A[i], 포인터 += 100
```

→ 루프 본체가 **단 2개 명령어**. `ADDIB,<`는 증가+비교+분기를 하나의 명령어로, `STWM`은 저장+포인터증가를 하나의 명령어로 수행한다.

최적화 컴파일러의 페이즈 구조

페이즈별로 순차 진행하며, 각 페이즈를 지날수록 코드 품질 개선. 서랍장처럼 페이즈 추가/삭제 용이.

```
Graph Representation of Intermediate Code
    ↓
(1)  Basic Block Optimization
(2)  Dataflow Analysis + Interval Analysis
```

```
(3)  Global Common Subexpression Elimination
(4)  Promotion of Memory Operations
(5)  Loop Invariant Code Motion
(6)  Induction Variable Analysis
(7)  Register Reassociation
(8)  Register Web (Live Range) Builder
(9)  Instruction Scheduling         ←── pre-Register Allocation
(10) Register Allocation             ←──
(11) Peephole Optimization
(12) Instruction Scheduling         ←── post-Register Allocation
    ↓
Graph Representation of Optimized Code
```

최적화 레벨: - `-O1`: 기본 최적화만 - `-O2` (= `-O`): 안정적인 최적화 - `-Ox` (x>2): 공격적이지만 항상 안정적이지는 않음

Bubble Sort 최적화 전/후 비교 (HP PA-RISC)

```c
#define N 100
main() {
    int A[N], i, j, temp;
    for (i = N-1; i >= 0; i--)
        for (j = 0; j < i; j++)
            if (A[j] > A[j+1]) {
                temp = A[j]; A[j] = A[j+1]; A[j+1] = temp;
            }
}
```

비최적화 코드 (inner loop): 약 50줄 어셈블리. 변수 i, j, temp를 매번 메모리(스택)에서 읽고 쓰고, 배열 주소(&A)를 매번 재계산(LDO -448(%r30)).

최적화 코드 (inner loop):

```
$006
LDWS    -4(%r23),%r25         ; A[j]
LDWS,MA 4(%r23),%r26         ; A[j+1], 포인터 자동 증가
COMB,<=,N %r25,%r26,$007     ; A[j] <= A[j+1]이면 swap 건너뜀
STWS    %r26,-8(%r23)        ; A[j+1] → A[j]
STWS    %r25,-4(%r23)        ; A[j] → A[j+1]
$007
ADDIB,<,N 1,%r24,$006+4     ; j++, j<i이면 $006으로
LDWS    -4(%r23),%r25        ; 다음 반복의 A[j] 미리 로드
```

→ 모든 변수가 레지스터에, 복합 명령어 사용, 루프가 극적으로 축소.

최적화 컴파일러의 문제 해결 방법론

1. 빈번한 케이스(common cases)를 찾아라
2. 수학적으로 정형화(formulate mathematically)하라
3. 적절한 페이즈를 결정하라
4. 알고리즘을 개발하라
5. 구현하라
6. 실제 데이터로 평가하라

> "케이스가 거의 없는 최적화에는 절대 시간을 쓰지 마라"

1.6 사례 연구: DRAM 테스트 패턴 프로그램 변환

배경: ATE(Automatic Test Equipment)는 ALPG(Algorithmic Pattern Generator)를 사용해 DRAM(DDR4 등)을 테스트한다. 각 클럭마다 패턴(command + data + address 비트 벡터)을 전송한다. 문제: ATE 장비 제조사마다 프로그래밍 언어가 다르고, 같은 장비의 다른 모델 간에도 HW(ALPG 개수, 레지스터 수)가 다르다.

해결: 통합 IR 기반 Translation Platform — 컴파일러와 동일한 구조 적용

```
[장비A 프로그램 TEST.asc]  → 분석(Analysis) → [통합 IR] → 합성
(Synthesis) → [장비B 프로그램 TEST.cs]
[장비A2 프로그램 TEST.asc2] →
→ [장비B2 프로그램 TEST.cs2]
```

Frontend: Lexing, Parsing, IR 생성, 인터프리터/VM. Backend: 불규칙 레지스터 할당, 다른 속도 DRAM 스케줄링, 가독성 중심 컴파일, 변환 검증, HW 설정 파일 자동 생성.

→ 컴파일러 기법이 프로그래밍 언어 번역을 넘어 다양한 분야에 범용적으로 적용 가능함을 보여주는 사례.

Ch2. Code Optimization Example — 단계별 최적화 실전

2.1 RISC CPU 개요 및 머신 코드 생성

RISC CPU 특성: - Uniform and orthogonal instruction set (단일 워드 명령어, opcode 위치 일정) - Identical general-purpose registers (동일한 범용 레지스터, 균일 사용) - Load/store-based architecture with simple addressing modes - → 쉬운 코드 생성과 최적화를 허용

RISC 명령어 형태:

```
Load:   load GP+offset, target-reg        (전역 변수 로드)
        load SP+offset, target-reg        (지역 변수 로드)
        loadi constant, target-reg        (상수 로드)
Compute: add src-reg1, src-reg2, tgt-reg  (레지스터 연산)
         ldo src-reg, constant, tgt-reg   (레지스터 + 상수)
Store:  store src-reg, GP+offset          (전역 변수 저장)
        store src-reg, SP+offset          (지역 변수 저장)
```

스택 머신 코드 → RISC 코드 변환:

스택의 각 슬롯에 의사 레지스터(pseudo register, s0, s1, ...) 부여. push→load, assign→store, 스택 깊이 추적.

`x = y` (x: 전역, y: 지역)의 변환 예:

```
push_const Glob+4     → loadi GP+4, s1           (x의 주소)
push_reg FP            → copy FP, s2
push_const 1, add      → loadi 1, s3; add s2,s3,s2
fetch                  → load s2(0), s2           (y의 값)
assign                 → store s1(0), s2          (y를 x에 저장)
```

결과: 의사 레지스터를 사용하는 **low-level IR** (물리 레지스터 미할당). 스택 머신 코드는 high-level IR.

2.2 예제 프로그램

```
int a[25][25];
main() {
    int i;
    for (i=0; i<25; i++)
        a[i][0] = 0;
}
```

메모리 공간과 레지스터: - 가상 주소 공간: code, global data, stack, heap - 32개 정수 레지스터: r0=항상 0, r27=GP(전역 데이터 포인터), r30=SP(스택 포인터) - 200 이상: 의사 레지스터 (미할당)

최종 최적화 어셈블리 (미리 보기):

```
ADDIL LR'a-$global$,%r27,%r1      ; &a 상위 21비트
LDO   RR'a-$global$(%r1),%r31    ; &a 하위 11비트
LDI   -25,%r23                     ; 카운터 = -25
$00000003
ADDIB,< 1,%r23,$00000003          ; 카운터++, <0이면 루프
STWM  %r0,100(%r31)               ; 0→a[i][0], 포인터+=100
```

2.3 비최적화 코드 (Un-optimized Code)

```
     STW    0,-40(30)                ; 0 → i (스택에 저장)
     LDW    -40(30),206              ; i를 r206에 로드
     LDI    25,212                   ; 25를 r212에 로드
     IFNOT  206 < 212 GOTO $02      ; i>=25이면 루프 탈출
     NOP
$03: LDW    -40(30),206              ; i를 r206에 로드  [중복 #1]
     ADDILG LR'a-$global$,27,213    ; &a 상위비트 (21bit + GP)
     LDO    RR'a-$global$(213),208  ; &a 하위비트 (11bit)
     MULTI  100,206,214              ; i × 100 (행크기: 25 int ×
4byte)
     ADD    208,214,215              ; &a + i×100 = &a[i][0]
     STWS   0,0(215)                 ; a[i][0] = 0
     LDW    -40(30),206              ; i를 r206에 로드  [중복 #2]
     LDO    1(206),216               ; i + 1
     STW    216,-40(30)              ; i = i+1 (스택에 저장)
     LDW    -40(30),206              ; i를 r206에 로드  [중복 #3]
     LDI    25,212                   ; 25를 r212에 로드  [중복]
     IF     206 < 212 GOTO $03      ; i<25이면 루프 반복
     NOP
$02:
```

주요 관찰: - 의사 레지스터(>200) 사용 중, r0/r27/r30만 실제 할당 - 스택 영역(r30 기반) vs 전역 데이터 영역(r27 기반) - ADDILG + LDO 쌍 = 32비트 주소 상수 획득 (SPARC의 sethi+or, MIPS의 lui+ori에 해당) - **MULTI 100**: a[25][25]에서 한 행 = 25 × 4byte = 100byte - `LDW -40(30),206`이 한 반복에 **4번** 등장 — 심각한 중복

2.4 단계 1: Basic Block Optimization

Basic Block (BB): 제어 흐름이 시작점에서 들어와 끝점에서 나가며, 중간에 분기가 없는 연속 명령어 시퀀스. BB 헤더 = 분기 타겟 또는 제어 합류점(join point).

BB 구축 방법: 명령어→다음 실행 명령어 그래프 구축 → BB 헤더 식별 → 두 헤더 사이의 명령어가 하나의 BB → BB들의 연결이 **CFG (Control Flow Graph)**.

예제의 CFG:

```
BB0 (초기화 + 첫 조건)
 │    STW 0,-40(30)
 │    LDW -40(30),206
 │    LDI 25,212
 │    IFNOT 206<212 GOTO $02
 │
 ↓          ↗ (루프백)
BB1 (루프 본체)
 │    LDW -40(30),206
 │    ADDILG ...
 │    LDO ...
 │    MULTI ...
 │    ADD ...
 │    STWS ...
 │    LDW -40(30),206
 │    LDO 1(206),216
 │    STW 216,-40(30)
 │    LDW -40(30),206
 │    LDI 25,212
 │    IF 206<212 GOTO $03
 │
 ↓
BB2 (루프 종료 후)
     Exit Code ...
```

지역(Local) 최적화 기법 — BB 내부에서 제어 흐름 정보 없이 수행:

(a) Load-to-Copy Optimization: store 직후 같은 위치 load → copy로 교체

```
변환 전: STW 0,-40(30); LDW -40(30),206
변환 후: STW 0,-40(30); COPY 0,206          ← 메모리 접근 → 레지스터 복사
```

(b) Local CSE (Common Subexpression Elimination): BB 내 동일 식 중복 계산 제거

```
변환 전: LDW -40(30),206  ... LDW -40(30),206  (같은 BB 내, r206 미변경)
변환 후: LDW -40(30),206  ... (삭제)
```

동일 식이 동일 타겟 레지스터를 가지면 두 번째 이후 삭제.

(c) Constant Folding: 컴파일 타임 계산 가능한 식을 상수로 교체

```
변환 전: if (4 > 3)
변환 후: if (true)
```

(d) Dead Code Elimination: 결과가 사용되지 않는 명령어 삭제

```
변환 전: x = y+1; ... x = 10   (첫 x 정의는 덮어쓰여져 사용 안 됨)
변환 후:        ; ... x = 10
```

(e) Copy Propagation / Constant Propagation: copy의 소스로 사용처 교체

```
변환 전: x = y; ... z = x+100
변환 후: x = y; ... z = y+100   (x 대신 y 사용 → x가 dead되면 삭제 가능)
```

핵심 포인트: 이런 최적화들은 "프로그래밍을 잘하면 불필요"해 보이지만, 컴파일러가 생성하는 코드에서는 **변수 읽기, 배열 주소 계산** 등에서 자연스럽게 발생한다. 예제에서 같은 메모리 로드와 계산이 반복되는 것이 그 증거.

지역 최적화 적용 후 코드:

```
     STW    0,-40(30)
     COPY   0,206                  ← load-copy (원래 LDW)
     LDI    25,212
                                   ← constant folding (IFNOT 삭제)
$03: LDW    -40(30),206
     ADDILG LR'a-$global$,27,213
     LDO    RR'a-$global$(213),208
     MULTI  100,206,214
     ADD    208,214,215
     STWS   0,0(215)
                                   ← CSE (LDW -40(30),206 삭제)
     LDO    1(206),216
     STW    216,-40(30)
     COPY   216,206                ← load-copy (원래 LDW)
                                   ← CSE (LDI 25,212 삭제, 위에서 이미
계산)
     IF     206 < 212 GOTO $03
     NOP
$02:
```

Extended Basic Block: 들어오는 분기 없이 나가는 분기만 있는 연속 BB 체인. 동일한 지역 최적화를 확장 적용 가능.

2.5 단계 2: Global CSE (전역 공통 부분식 제거)

Available Expression: 지점 p에 도달하는 **모든 경로**에서 식 x+y가 계산되었고, 마지막 계산 이후 x나 y에 대한 재대입이 없으면 p에서 x+y가 "available"하다. p에서 x+y를 다시 계산하면 중복 → 제거.

가정: Value Numbering — 코드 생성기가 동일 계산(RHS)에 동일 타겟 의사 레지스터를 부여. 예: `LDW -40(30),206`은 항상 r206에, `LDI 25,212`는 항상 r212에.

적용 결과: BB0에서 이미 계산한 값이 BB1 시작점에서 available → BB1 내 중복 삭제:

```
     STW    0,-40(30)
     COPY   0,206
     LDI    25,212
$03:                               ← available: r206(i값), r212(25)
     ADDILG LR'a-$global$,27,213  ← LDW -40(30),206 삭제됨
     LDO    RR'a-$global$(213),208
     MULTI  100,206,214
     ADD    208,214,215
     STWS   0,0(215)
     LDO    1(206),216
     STW    216,-40(30)
     COPY   216,206
     IF     206 < 212 GOTO $03    ← LDI 25,212 삭제됨
     NOP
```

2.6 Definition, Use, Live Range

Definition (정의) = 쓰기(write). **Use** (사용) = 읽기(read). - `x = y + z`에서: 1개 정의(x), 2개 사용(y, z)

Live Range = 서로 "도달(reach)"할 수 있는 정의와 사용의 집합. 같은 저장 위치(레지스터 또는 메모리)에 접근.

```
x = load     ─┐
y = x/2        ├── Live Range 1 (x의 첫 번째 정의와 그 사용)
               │
x = z+1      ─┘─┐
x = 10          ├── Live Range 2 (x의 두 번째/세 번째 정의와 사용)
...             │
w = x+1        ─┘
```

두 종류: - **Register live range (web)**: 레지스터 할당의 단위 - **Memory live range**: 같은 메모리 위치에 접근

2.7 단계 3: Register Promotion (레지스터 승격)

목적: 메모리 연산(load/store)을 레지스터 연산(copy)으로 승격.

절차: 1. Memory Live Range 구축 = 같은 메모리 위치에 접근하는 store/load 집합 2. 주소 유도(address derivation) 계산: `STW 0,-40(30)` → 주소 = -40 + SP(r30) 3. 조건 확인: **singleton 변수**여야 함 (배열, 포인터, 구조체 불가) 4. 변환: **store → copy**, **load → 삭제**

프론트엔드가 같은 위치 접근 정보를 제공하면 좋고, 아니면 백엔드가 코드 분석으로 찾아야 함.

예제 적용: 변수 i에 대한 2개의 Memory Live Range:

```
MLR1: STW 0,-40(30) → COPY 0,206        (store→copy)
      COPY 0,206    → (이미 copy)         (load 이미 삭제)
MLR2: STW 216,-40(30) → COPY 216,206    (store→copy)
      COPY 216,206    → (이미 copy)       (load 이미 삭제)
```

Register Promotion 후 코드:

```
     COPY   0,206                 ; i = 0 (레지스터에!)
     LDI    25,212
     NOP
$03: ADDILG LR'a-$global$,27,213
     LDO    RR'a-$global$(213),208
     MULTI  100,206,214
     ADD    208,214,215
     STWS   0,0(215)              ; a[i][0] = 0 (이것은 배열이므로 승격
불가)
$01: LDO    1(206),216            ; i + 1
     COPY   216,206               ; i = i + 1
     IF     206 < 212 GOTO $03
     NOP
```

→ 변수 i에 대한 **모든 STW, LDW가 사라졌다!** 배열 a에 대한 STWS는 singleton이 아니므로 유지.

2.8 단계 4: Loop Optimization

세 가지 전통적 루프 최적화:

(a) LICM (Loop Invariant Code Motion): 루프 안에서 매 반복 같은 결과인 코드를 루프 밖(헤더)으로 이동

대상: `ADDILG LR'a-$global$,27,213` + `LDO RR'a-$global$(213),208` — 배열 a의 기본 주소는 매 반복 동일 → 루프 전에 한 번만 계산.

(b) Strength Reduction (강도 감소): 비싼 연산(곱셈)을 싼 연산(덧셈)으로 교체

유도 변수 i가 0,1,2,...,24로 변할 때:

```
원래: MULTI 100, i, offset        ; offset = i × 100 (곱셈, 매 반복)
변환: 새 변수 r218 도입
      초기: LDI 0,218              ; r218 = 0
      루프: LDO 100(218),220       ; r218 + 100 (덧셈, 매 반복)
            COPY 220,218           ; r218 = r218 + 100
```

(c) Induction Variable Elimination (유도 변수 제거): 강도 감소 후 원래 유도 변수가 루프 조건에만 쓰이면, 새 변수로 조건도 교체하여 원래 변수 제거

```
원래: IF 206 < 212 GOTO $03       ; i < 25
변환: LDI 2500,219                 ; 25 × 100 = 2500
      IF 218 < 219 GOTO $03       ; offset < 2500
```

→ 원래 유도 변수 i(r206)와 상수 25(r212)가 **불필요**해짐.

루프 최적화 후 코드:

```
     COPY   0,206                  ;  (나중에 dead code)
     LDI    25,212                 ;  (나중에 dead code)
     ADDILG LR'a-$global$,27,213  ← LICM: 루프 밖으로
     LDO    RR'a-$global$(213),208 ← LICM: 루프 밖으로
     LDI    0,218                  ← 새 유도변수 초기화
     LDI    2500,219               ← 새 상한값
$03:
     COPY   218,214                ← MULTI 100,206,214 대체
     ADD    208,214,215
     STWS   0,0(215)
     LDO    100(218),220           ← LDO 1(206),216 대체 (덧셈)
     COPY   220,218                ← COPY 216,206 대체
     IF     218 < 219 GOTO $03    ← IF 206 < 212 대체
```

2.9 단계 5: Building Register Live Ranges (Webs) + Dead Code Elimination

Register Live Range(Web) = 레지스터 정의와 사용의 도달 가능 집합. **레지스터 할당의 단위**. 또한 Dead Code를 식별하는 데 도움 — 어떤 사용에도 도달하지 않는 정의는 dead.

Dead instructions 발견: - `COPY 0,206` — r206은 이제 루프 안에서 사용되지 않음 (유도 변수가 r218로 교체됨) - `LDI 25,212` — r212도 더 이상 사용되지 않음

이 두 명령어 삭제. 의사 레지스터 번호를 web 번호로 교체:

```
     ADDILG LR'a-$global$,27,66
     LDO    RR'a-$global$(66),65
     LDI    0,69
     LDI    2500,71
$03:
     COPY   69,67
     ADD    65,67,68
     STWS   0,0(68)
     LDO    100(69),70
     COPY   70,69
     IF     69 < 71 GOTO $03
```

2.10 단계 6: Instruction Scheduling (명령어 스케줄링)

가정: 머신에 ALU 2개 → 한 클럭에 2개 명령어 동시 실행 가능.

원리: 서로 독립적인(데이터 의존성 없는) 명령어를 같은 사이클에 배치. 의존성이 있는 명령어 사이에 독립 명령어를 끼워넣음.

적용:

```
스케줄 전:           스케줄 후:
COPY 69,67           COPY 69,67
ADD  65,67,68        LDO  100(69),70    ← code motion (독립적)
STWS 0,0(68)         ADD  65,67,68
LDO  100(69),70      COPY 70,69         ← code motion
COPY 70,69           STWS 0,0(68)
IF   69<71 ...       IF   69<71 ...
```

`LDO 100(69),70`은 r69만 읽으므로, `ADD`가 쓰는 r67, r68과 무관 → 앞으로 이동 가능.

2.11 단계 7: Register Allocation + Copy Elimination

Graph Coloring 기반 레지스터 할당: 1. **Interference Graph** 구축: 노드=live range(web), 간선=동시에 살아있는 쌍 2. 물리 레지스터 수만큼의 색으로 그래프 컬러링 3. NP-complete → **휴리스틱** 사용 4. 레지스터 부족 시 **Spilling**: 일부 web을 스택 메모리로 (def 직후 store, use 직전 load 추가)

두 web이 간섭하는 예: `x = RHS1; y = RHS2; z = x*10; ... = y/x` → x와 y가 동시에 live → 같은 레지스터 불가

또한 **caller-save / callee-save 레지스터** 코드 생성 필요.

Copy Elimination — 두 가지 접근:

(a) Copy Propagation: copy를 dead code로 만듦

```
y=x; z=y+1 → z=x+1   (y 대신 x 사용 → y=x가 dead되면 삭제)
```

(b) Copy Coalescing: copy의 소스와 타겟 live range가 **간섭하지 않으면** 두 web을 합쳐 같은 레지스터 할당 → `COPY rN, rN` 형태가 되어 삭제 - **부작용**: 합친 노드의 간선이 합집합(union of edges) → 컬러링 어려워짐

예제 적용: - `COPY 69,67`: 67의 모든 사용을 69로 교체 → copy propagation → 삭제 - `COPY 70,69`: web 70과 69가 간섭 안 함 → coalescing → `COPY 69,69` → 삭제

최종 코드 (non-scheduled version):

```
     ADDILG LR'a-$global$,27,66
     LDO    RR'a-$global$(66),65
     LDI    0,69
     LDI    2500,71
$03:
     ADD    65,69,68            ; &a + offset
     STWS   0,0(68)             ; a[i][0] = 0
     LDO    100(69),69          ; offset += 100
     IF     69 < 71 GOTO $03   ; offset < 2500?
```

→ **루프 본체 4개 명령어**. 원래 13개에서 극적 축소.

참고: scheduled version에서는 coalescing이 불가능할 수 있다. 또한 compound instructions를 생성하면 더 축소 가능.

2.12 SSA Form (Static Single Assignment)

정의: 각 변수가 프로그램 전체에서 **단 한 번만 정의**되는 표현 형태. live range renaming과 유사.

현대 컴파일러의 표준 표현: Normal form → SSA form → 분석/최적화 → Normal form

```
일반 형태:            SSA 형태:
a = x + y             a0 = x + y
b = a - 1             b0 = a0 - 1
a = y + b             a1 = y + b0      ← 다른 이름
b = x * 4             b1 = x * 4       ← 다른 이름
a = a + b             a2 = a1 + b1
```

φ-function (파이 함수): 제어 합류점(join point)에 삽입. 어느 경로로 왔느냐에 따라 값을 선택. **실제로 실행되지 않는 표기법(notational fiction)** — 분석과 최적화를 위한 것.

```
일반 형태:            SSA 형태:
b = M[x]              b1 = M[x0]
a = 0                 a1 = 0
if b < 4              if b1 < 4
  a = b                 a2 = b1
                      a3 = φ(a2, a1)    ← 합류점
c = a + b             c1 = a3 + b1
```

SSA의 이점 — 상수 전파(Constant Propagation) 예:

```
일반 형태:                      SSA 형태:
x = 1                           x0 = 1
y = 2                           y = 2
z = x + y   ← 어떤 x?         z = x0 + y     ← x0=1 확실!
if (...)                        if (...)
h = x * 2   ← 어떤 x?         h = x0 * 2     ← x0=1 확실!
x = f(...)                      x1 = f(...)
k = x - y   ← 어떤 x?         x2 = φ(x0, x1)
                                 k = x2 - y     ← x2는 φ, 상수 아님
```

일반 형태에서는 각 x 사용이 어느 정의에서 왔는지 별도 분석 필요. SSA에서는 **변수 이름만 보면 바로 확인** 가능.

Ch3. Data Flow Analysis — 데이터 흐름 분석

3.1 데이터 흐름 분석이란?

주어진 함수가 데이터를 어떻게 다루는지에 대한 **전역적 정보(global information)**를 계산하고, **각 명령어 경계(instruction boundary)**마다 그 정보를 제공하는 것.

용도 (최적화마다 다른 정보 필요): - Global CSE → **Available Expressions** (사용 가능한 표현식) - Register Live Range 구축 → **Reaching Definitions** (도달 정의) - Code Motion → **Live Variables** (생존 변수) - Constant Folding → 어떤 변수가 상수인지, 그 값이 무엇인지

핵심 원칙: 잘못된 정보 → 잘못된 최적화. 정확하지 않으면 **보수적 근사 (conservative approximation)** 필요. 최적화 기회를 놓치는 건 OK, 프로그램 의미를 바꾸면 절대 안 됨.

3.2 분석 전략: 2단계

```
1단계: Global Analysis — CFG 위에서 BB 단위, 각 BB 경계의 정보 계산
       BB의 효과를 하나의 명령어처럼 요약하여 사용

2단계: Local Analysis — BB 내부, 각 명령어 경계의 정보 계산
       전역 분석 결과(BB 경계 정보)를 출발점으로, 명령어 효과를 순차 적용
```

Local Analysis 예시 (Reaching Definitions):

```
BB 시작:                    도달 정의 = {d1, d2}  (전역 분석에서 제공)
x = 1 (d3)     → kill d1   도달 정의 = {d2, d3}
y = 1 (d4)     → kill d2   도달 정의 = {d3, d4}
z = x+y (d5)               도달 정의 = {d3, d4, d5}

이 BB의 요약: KILL={d1,d2}, GEN={d3,d4}  (d5는 z의 마지막 정의이므로 GEN
에 포함되어야 하지만 d3,d4가 x,y의 마지막 정의)
```

3.3 명령어의 효과 (Effect of an Instruction)

`a = b + c`의 경우: - **Uses**: 변수 b와 c - **Kills**: a에 대한 이전 정의 (a를 새로 정의하므로) - **Generates**: a에 대한 새 정의

이 효과를 명령어에 적용하여 데이터 흐름 정보를 "통과(thru)" 전파.

3.4 기본 블록의 효과 (Effect of a BB)

BB 내 명령어 효과를 **합성(compose)** → BB 전체의 요약.

핵심 3개념:

(1) Locally Exposed Use: BB 내에서 변수를 사용하는데, **그 사용 전에 같은 변수의 정의가 BB 내에 없는** 경우. 즉 BB 외부에서 들어온 값을 사용.

(2) Kill: BB 내에서 변수를 정의하면, 그 변수에 대한 **BB에 도달하는 모든 이전 정의**를 kill.

(3) Locally Generated Definition: BB 내에서 변수에 대한 **마지막 정의**.

구체 예:

```
r4 = r1 + r2        ; r1 사용(exposed), r2 사용(exposed). r4 정의.
r2 = r4 + 1         ; r4 사용(NOT exposed, 위에서 정의됨). r2 재정의.
r5 = r3 + r1        ; r3 사용(exposed), r1 사용(exposed). r5 정의.
r1 = r5             ; r5 사용(NOT exposed). r1 재정의.
r4 = r2 * r4        ; r2, r4 사용(NOT exposed). r4 재정의.
r2 = r5             ; r5 사용(NOT exposed). r2 재정의.
if r2 > 100 goto L1 ; r2 사용(NOT exposed).
```

- **Locally exposed uses**: {r1, r2, r3} — BB 진입 시 값이 필요
- **Locally generated defs**: r1(=r5), r4(=r2*r4), r2(=r5) — 각 변수의 마지막 정의
- Kill: r4, r2, r1에 대한 이전(BB 외부) 정의들

3.5 정적 프로그램 vs 동적 실행 (Static vs Dynamic)

- **정적**: 유한한 프로그램 코드
- **동적**: 잠재적으로 무한한 실행 경로 (루프 때문)

데이터 흐름 분석의 정의: CFG의 각 **정적 지점**에, 그 지점에 도달하는(또는 출발하는) **모든 동적 실행 경로**에 대해 참인 정보를 연관짓는 것.

방법: ① 각 경로의 BB 효과를 합성 → ② 진입점(또는 출구)의 정보에 적용 → ③ 모든 경로의 결과를 합쳐(merge) → 해당 지점의 정보.

루프가 있으면 경로가 무한 → **반복(iteration)으로 고정점(fixed point)까지 수렴**.

3.6 Reaching Definitions (도달 정의) 분석

정의

- **Definition**: 변수 x에 값을 대입하는(또는 대입할 수 있는, e.g. 조건 실행) 명령어.
- **Reach**: 정의 d가 지점 p에 도달한다 = d 직후부터 p까지의 경로가 **존재**하고(∃), 그 경로에서 d가 **kill되지 않음**.

```
         d1: a = 10
         d2: b = 11
         if e
        /       \
d3: a = 1     d5: c = a     ← a의 값은 d1에서 옴
d4: b = 2     d6: a = 4
        \       /
         합류점         ← a의 reaching defs = {d3, d6} (d1은 양쪽 모두 kill)
                          b의 reaching defs = {d4, d2} (d2는 왼쪽에서 kill, 오른쪽에서 살아남)
```

문제 정의

각 BB b에 대해: - **IN[b]**: b의 시작점에 도달하는 정의들의 집합 - **OUT[b]**: b의 끝점에서 나가는 정의들의 집합 - 표현: **비트 벡터** (함수 내 정의 하나당 1비트)

전달 함수 (Transfer Function) — Forward

IN[b]가 주어졌을 때 OUT[b]를 계산하는 것이 자연스럽다 (forward problem).

왜? 만약 OUT[b]에서 IN[b]를 역으로 구하려면, OUT[b]에 포함된 정의 d가 b 내부에서 정의된 것인 경우 IN[b]에 포함해야 하는지 불분명. → forward가 자연스러움.

```
OUT[b] = GEN[b] ∪ (IN[b] − KILL[b])
```

- **GEN[b]**: BB b에서 새로 생성된 정의 (locally generated definitions)
- **KILL[b]**: 함수 내에서 b의 정의들과 **같은 변수를 정의하는 다른 명령어**들의 집합
- **IN[b] − KILL[b]**: b에 들어온 정의 중, b에서 kill되지 않고 살아남은 것

KILL의 정확한 의미 (주의!)

KILL[b]는 BB b의 **정적 속성(static property)**이다. "만약 이 정의가 b에 도달한다면 b를 통과하면서 무효화된다"는 **잠재적 능력**을 기술. 실제로 kill이 발생하려면 해당 정의가 IN[b]에 포함되어 있어야 한다.

예: B1이 함수 시작 → IN[B1] = {} → KILL[B1]에 무엇이 있든 `{} - KILL = {}` → 효과 없음.

KILL은 "미래 정의를 죽인다"가 아니라 "같은 변수의 다른 정의 목록"이라는 정적 정보.

합류 연산 (Meet Operator) — Edges

단일 predecessor: `IN[b] = OUT[p]` (그대로 전파)

Join node (여러 predecessor): 어떤 경로든 **하나라도** 도달 가능하면 포함 → **합집합(union)**

```
IN[b] = OUT[p1] ∪ OUT[p2] ∪ ... ∪ OUT[pn]
```

Cyclic Graph (루프)에서의 처리

방정식은 동일: `OUT[b] = GEN[b] ∪ (IN[b] - KILL[b])`, `IN[b] = ∪ OUT[pred]`

해: 방정식을 만족하는 **고정점(fixed point) 해**. 반복 계산으로 구한다.

Worklist Algorithm

```
/* Initialize */
OUT[Entry] = {}
for all nodes i: OUT[i] = {}
ChangeNodes = N (모든 노드)

/* Iterate */
while ChangeNodes != {} {
    remove i from ChangeNodes
    IN[i] = ∪ OUT[p], for all predecessors p of i
    oldout = OUT[i]
    OUT[i] = GEN[i] ∪ (IN[i] - KILL[i])
    if (oldout != OUT[i]) {
        for all successors s of i:
            add s to ChangeNodes
    }
}
```

예시 문제 풀이 — Reaching Definitions

CFG:

```
entry
  ↓
B1: d1: i=m-1,  d2: j=n,    d3: a=u1
  ↓
B2: d4: i=i+1,  d5: j=j-1     ←── B3에서 루프백
  ↓               ↘
B3: d6: a=u2    B4: d7: i=u3
  (→B2 루프백)      ↓
                  exit
```

Step 1: GEN/KILL 계산

GEN[b] = BB 내 각 변수의 마지막 정의 모음 KILL[b] = BB 내 정의된 각 변수에 대해, 함수 전체에서 같은 변수를 정의하는 **다른** 명령어들

BB	정의	GEN	KILL (설명)
B1	d1(i), d2(j), d3(a)	{1,2,3}	{4,5,6,7} — i→{d4,d7}, j→{d5}, a→{d6}
B2	d4(i), d5(j)	{4,5}	{1,2,7} — i→{d1,d7}, j→{d2}
B3	d6(a)	{6}	{3} — a→{d3}
B4	d7(i)	{7}	{1,4} — i→{d1,d4}

Step 2: 반복 계산

초기: 모든 OUT = {}

1회차:

```
B1: IN = OUT[entry] = {}
    OUT = {1,2,3} ∪ ({} − {4,5,6,7}) = {1,2,3}

B2: IN = OUT[B1] ∪ OUT[B3] = {1,2,3} ∪ {} = {1,2,3}
    OUT = {4,5} ∪ ({1,2,3} − {1,2,7}) = {4,5} ∪ {3} = {3,4,5}

B3: IN = OUT[B2] = {3,4,5}
    OUT = {6} ∪ ({3,4,5} − {3}) = {6} ∪ {4,5} = {4,5,6}

B4: IN = OUT[B2] = {3,4,5}
    OUT = {7} ∪ ({3,4,5} − {1,4}) = {7} ∪ {3,5} = {3,5,7}
```

2회차 (B2의 predecessor B3의 OUT이 변화했으므로):

```
B2: IN = OUT[B1] ∪ OUT[B3] = {1,2,3} ∪ {4,5,6} = {1,2,3,4,5,6}  ← 변화!
    OUT = {4,5} ∪ ({1,2,3,4,5,6} − {1,2,7}) = {4,5} ∪ {3,4,5,6} = {3,4,5,6}  ← 변화!

B3: IN = OUT[B2] = {3,4,5,6}
    OUT = {6} ∪ ({3,4,5,6} − {3}) = {6} ∪ {4,5,6} = {4,5,6}  ← 변화 없음

B4: IN = OUT[B2] = {3,4,5,6}
    OUT = {7} ∪ ({3,4,5,6} − {1,4}) = {7} ∪ {3,5,6} = {3,5,6,7}  ← 변화!
```

3회차: B2 재확인 → IN = {1,2,3} ∪ {4,5,6} = {1,2,3,4,5,6} → OUT = {3,4,5,6} → 변화 없음 → **수렴!**

최종 결과:

```
       IN                  OUT
entry: {}                  {}
B1:    {}                  {1,2,3}
B2:    {1,2,3,4,5,6}       {3,4,5,6}
B3:    {3,4,5,6}           {4,5,6}
B4:    {3,4,5,6}           {3,5,6,7}
```

해석: B2 시작점에서 IN={1,2,3,4,5,6} — d1~d6 모두 도달 가능. B1에서 직접(d1,d2,d3) + B3 루프백(d4,d5,d6). d7은 B2에 도달하지 않음(B4는 B2의 predecessor가 아님).

3.7 Live Variables (생존 변수) 분석

정의

- **Live**: 변수 v가 지점 p에서 live = p에서 시작하는 **어떤 경로**를 따라 v의 값이 (재정의 전에) 사용됨
- **Dead**: p 이후 어떤 경로에서도 v가 (재정의 전에) 사용되지 않음

용도: 레지스터 할당 (dead 변수의 레지스터를 다른 용도로 사용 가능), Dead Code Elimination.

전달 함수 — Backward

OUT[b]가 주어졌을 때 IN[b]를 계산 (**backward problem**):

```
IN[b] = USE[b] ∪ (OUT[b] − DEF[b])
```

- **USE[b]**: BB b의 locally exposed uses (사용 전에 BB 내 정의가 없는 변수)
- **DEF[b]**: BB b에서 정의된 변수들의 집합
- **OUT[b] − DEF[b]**: b 이후에 live한 변수 중, b에서 재정의되지 않은 것

합류 연산

Live Variables의 "합류 노드" = **여러 successor를 가진 노드** (분기점). 정보가 뒤에서 앞으로 흐르므로.

```
OUT[b] = IN[s1] ∪ IN[s2] ∪ ... ∪ IN[sn]   (모든 successor s)
```

합집합(union): 어떤 후속 경로에서든 사용될 가능성이 있으면 live.

Worklist Algorithm (Backward)

```
/* Initialize */
IN[Exit] = {}
for all nodes i: IN[i] = {}
ChangeNodes = N

/* Iterate */
while ChangeNodes != {} {
    remove i from ChangeNodes
    OUT[i] = ∪ IN[s], for all successors s of i
    oldin = IN[i]
    IN[i] = USE[i] ∪ (OUT[i] − DEF[i])
    if (oldin != IN[i]) {
        for all predecessors p of i:
            add p to ChangeNodes      ← RD와 반대 방향!
    }
}
```

3.8 Framework 대비표

항목	Reaching Definitions	Live Variables
Domain	Sets of **Definitions**	Sets of **Variables**
Transfer Function f_b(x)	GEN[b] ∪ (x − KILL[b])	USE[b] ∪ (x − DEF[b])

Direction	**Forward**: OUT[b] = f_b(IN[b])	**Backward**: IN[b] = f_b(OUT[b])
Generate	GEN[b]: BB 내 정의	USE[b]: BB 내 사용
Propagate	IN[b] − KILL[b]	OUT[b] − DEF[b]
Merge Operation	IN[b] = ∪ OUT[pred]	OUT[b] = ∪ IN[succ]
Initialization	OUT[b] = {}	IN[b] = {}
Boundary Condition	OUT[entry] = {}	IN[exit] = {}
변화 시 전파 대상	**successor** 추가	**predecessor** 추가

두 분석은 완벽하게 대칭적이며, 이 구조가 Ch4(Foundations of Dataflow Analysis)의 일반화된 프레임워크(격자 이론, 단조 함수)의 기초가 된다.